

Bisection vs Illinois:

A Comparison of Root-finding Methods

Matt D'Urso

Aug 3, 2026

Colgate University

These slides cover 3 root-finding methods:

1. Bisection
2. Regula falsi “method of false position”
3. Illinois method (Snyder, 1953; Dowell & Jarratt, 1971)

I will assume you know the concept of bisection, so that will be a quick review/reference point for us

Why even do this?-

Well, why even do anything? If you're going to do something, do it well.

More importantly, when it's 3:30am and you're trying to speed through a calibration, each iteration takes years off your life and decades off your rapidly dwindling sanity

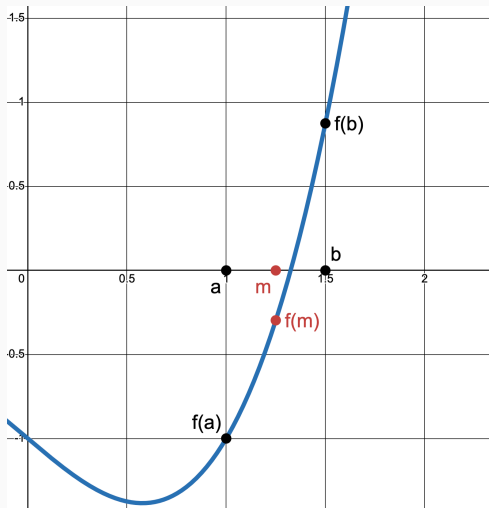
- You'd like to shrink the initial brackets, but when targeting strong asymmetries (e.g. size distributions), things don't behave as nicely as you'd like
- Widening the brackets adds 1, maybe 2 iters, but you are no longer a sane person that properly allocates your time

Suppose I wanted to find the root of $f(x)$, which is continuous between a and b :

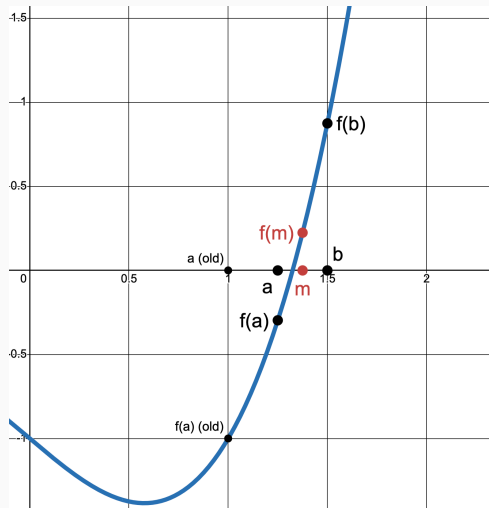
1. Pick initial brackets (a and b) where $f(a)$ and $f(b)$ have opposite signs
2. Find the midpoint of a and b , $m = \frac{a+b}{2}$ and take $f(m)$
3. If $f(m)$ has the same sign as $f(a)$, replace a with m
 - Or, replace b with m if $f(m)$ has the same sign as $f(b)$
4. Continue until $|f(m)| < \text{desired_tol}$. Then m sufficiently approximates the root

Next slide: example with $f(x) = x^3 - x - 1$ and initial $a = 1$ and $b = 1.5$

Bisection: Steps 1-3 (All graphs made with Desmos)



Matt D'Urso



Illinois Method

Bisection: Steps 4 - N

Now $m = 1.375$ becomes the new b and the process repeats until $f(m)$ is close to 0

- Determined by $|f(m)| < \epsilon$, where ϵ is sufficiently small and chosen by the modeler

This is a perfectly valid method, but we can make it faster with the “false position”

We use bisection, or various other numerical methods, because economics is hard

- What is simple is finding the root of a line

We can use the root of a line to better update our new a or b point:

1. Instead of the midpoint of a and b , draw a line between $f(a)$ and $f(b)$
2. Call the root of this line c and take $f(c)$
 - I'm using c since this is no longer a midpoint, but call it m if you like
3. If $f(c)$ has the same sign as $f(a)$, replace a with c
 - Or, replace b with c if $f(c)$ has the same sign as $f(b)$
4. Continue until $|f(c)| < \text{desired_tol}$. Then c sufficiently approximates the root

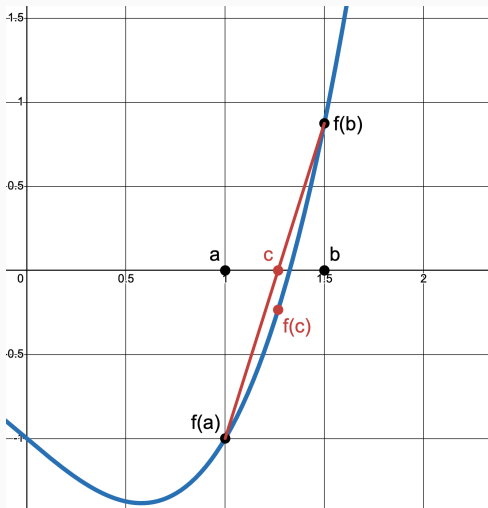
To draw our line, linearly interpolate (and rearrange a bit) $f(a)$ and $f(b)$:

$$c = b - \frac{f(b) \times (b - a)}{f(b) - f(a)}$$

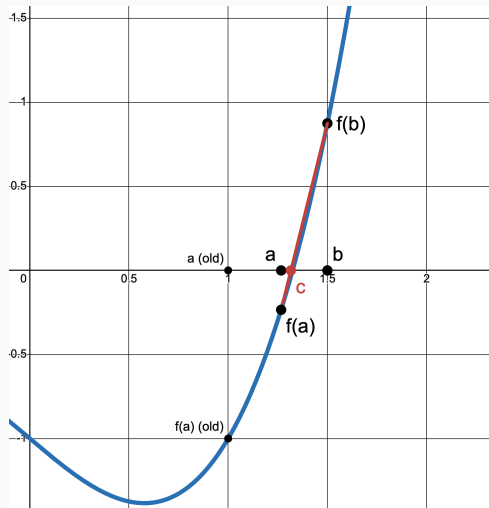
Optional: set a condition to use the smaller $|f(val)|$. If $|f(a)| < |f(b)|$, use:

$$c = a - \frac{f(a) \times (b - a)}{f(b) - f(a)}$$

Regula Falsi: Steps 1-4



Matt D'Urso



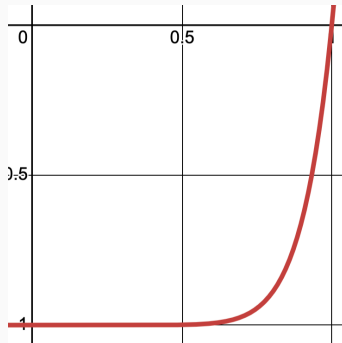
Illinois Method

8/13

Issues?

Consider the case of $f(x) = x^{10} - 1$ on $[0, 1.3]$

- $f(x)$ is flat near 0 and blows up near 1.3
- Our line always crosses the axis around the left end
- $b = 1.3$ is retained forever and a crawls towards 1 (true root)



The Illinois Method

The Illinois method aims to correct this issue

- First as technical note by Snyder (1953), then formalized by Dowell & Jarratt (1971)

If $\text{sign } f(c) = \text{sign } f(a)$, then the root is in $[c, b]$

- c is the new a ; b remains the same

If this happens twice in a row, assign $f(b) = \frac{f(b)}{2}$ (same for $f(a)$ if opposite case)

- If it happens again on the next iter, halve again
- *Everything else remains the same as in regula falsi*
- Convergence guaranteed (halving does not change sign; stale bracket replaced)
 - ▷ Order of convergence ≈ 1.442 (Dowell & Jarratt, 1971)

Regula Falsi Failure and Illinois Correction for $f(x) = x^{10} - 1$

Regula Falsi:

```
iter=1 a=0.0 b=1.3 c=0.0942995953723274
iter=2 a=0.0942995953723274 b=1.3 c=0.18175887251907943
iter=3 a=0.18175887251907943 b=1.3 c=0.26287401252030435
iter=4 a=0.26287401252030435 b=1.3 c=0.3381051033222695
iter=5 a=0.3381051033222695 b=1.3 c=0.4078779165927524
iter=6 a=0.4078779165927524 b=1.3 c=0.47258315356239
iter=7 a=0.47258315356239 b=1.3 c=0.5325715106320142
iter=8 a=0.5325715106320142 b=1.3 c=0.5881445691706799
iter=9 a=0.5881445691706799 b=1.3 c=0.6395439707682159
iter=10 a=0.6395439707682159 b=1.3 c=0.6869431667412391
iter=11 a=0.6869431667412391 b=1.3 c=0.7304464366929381
iter=12 a=0.7304464366929381 b=1.3 c=0.7700987444968075
iter=13 a=0.7700987444968075 b=1.3 c=0.8059075090169241
iter=14 a=0.8059075090169241 b=1.3 c=0.8378738914244527
iter=15 a=0.8378738914244527 b=1.3 c=0.8660276320873561
iter=16 a=0.8660276320873561 b=1.3 c=0.8904572813509999
iter=17 a=0.8904572813509999 b=1.3 c=0.911328251308228
iter=18 a=0.911328251308228 b=1.3 c=0.9288846695992143
iter=19 a=0.9288846695992143 b=1.3 c=0.9434360517492835
iter=20 a=0.9434360517492835 b=1.3 c=0.9553339751921469
iter=21 a=0.9553339751921469 b=1.3 c=0.9649455723416893
iter=22 a=0.9649455723416893 b=1.3 c=0.9726296888054463
iter=23 a=0.9726296888054463 b=1.3 c=0.9787191336334624
iter=24 a=0.9787191336334624 b=1.3 c=0.982568853336012
```

Max iter hit at 100

Illinois:

```
iter=1 a=0.0 b=1.3 c=0.0942995953723274
iter=2 a=0.0942995953723274 b=1.3 c=0.18175887251907943
iter=3 a=0.18175887251907943 b=1.3 c=0.3330171567671213
iter=4 a=0.3330171567671213 b=1.3 c=0.5634423147022629
iter=5 a=0.5634423147022629 b=1.3 c=0.8463635731395354
iter=6 a=0.8463635731395354 b=1.3 c=1.074910177068493
iter=7 a=0.8463635731395354 b=1.074910177068493 c=0.9454923183277778
iter=8 a=0.9454923183277778 b=1.074910177068493 c=0.9828011093189349
iter=9 a=0.9828011093189349 b=1.074910177068493 c=1.0040954923602121
iter=10 a=0.9828011093189349 b=1.0040954923602121 c=0.9996755374546944
iter=11 a=0.9996755374546944 b=1.0040954923602121 c=0.9999940615677148
iter=12 a=0.9999940615677148 b=1.0040954923602121 c=1.000005704633256
iter=13 a=0.9999940615677148 b=1.000005704633256 c=0.9999999998475554
iter=14 a=0.9999999998475554 b=1.000005704633256 c=0.9999999999999961
```

My JMP D-StSt: Bisection vs Illinois

Bisection:

METHOD 1: BISECTION				
bracket [5, 10], tol = 1e-08				
iter	pval	wage	C	f_dist
1	5.0000000	0.428000	0.030432	-1.70e-01
2	10.0000000	0.214000	0.568592	+4.69e-01
3	7.5000000	0.285333	0.167265	+3.39e-02
4	6.2500000	0.342400	0.077432	-8.26e-02
5	6.8750000	0.311273	0.115721	-2.97e-02
6	7.1875000	0.297739	0.139706	+5.76e-04
7	7.0312500	0.304356	0.127275	-1.49e-02
8	7.1093750	0.301011	0.133378	-7.28e-03
9	7.1484375	0.299366	0.136514	-3.38e-03
10	7.1679688	0.298550	0.138103	-1.41e-03
11	7.1777344	0.298144	0.138903	-4.17e-04
12	7.1826172	0.297942	0.139304	+7.92e-05
13	7.1801758	0.298043	0.139104	-1.69e-04
14	7.1813965	0.297992	0.139204	-4.48e-05
15	7.1820068	0.297967	0.139254	+1.72e-05
16	7.1817017	0.297980	0.139229	-1.38e-05
17	7.1818542	0.297973	0.139242	+1.70e-06
18	7.1817780	0.297976	0.139235	-6.06e-06
19	7.1818161	0.297975	0.139238	-2.18e-06
20	7.1818352	0.297974	0.139240	-2.43e-07
21	7.1818447	0.297974	0.139241	+7.26e-07
22	7.1818399	0.297974	0.139240	+2.42e-07
23	7.1818376	0.297974	0.139240	-7.39e-10

METHOD 1: BISECTION: p = 7.1818376 23 solves total 146.48 seconds

(Using $\text{xtol} (|b - a| < \epsilon)$ makes the difference even stronger)

Illinois:

METHOD 2: ILLINOIS				
bracket [5, 10], tol = 1e-08				
iter	pval	wage	C	f_dist
1	5.0000000	0.428000	0.030432	-1.70e-01
2	10.0000000	0.214000	0.568592	+4.69e-01
3	6.3285719	0.338149	0.081606	-7.64e-02
4	6.8432961	0.312715	0.113480	-3.26e-02
5	7.2293715	0.296015	0.143187	+4.86e-03
6	7.1793218	0.298078	0.139033	-2.56e-04
7	7.1818209	0.297975	0.139239	-1.70e-06
8	7.1818540	0.297973	0.139241	+1.67e-06
9	7.1818376	0.297974	0.139240	-6.44e-12

METHOD 2: ILLINOIS: p = 7.1818376 9 solves total 57.87 seconds

Other Methods?

Many other weights (or functions of weights) exist, along with a multitude of completely different methods, which may be faster

The Illinois method has three benefits in one: speed, [relative] ease of learning, and guaranteed to work (again, on a continuous on a sign-change bracket)

References

-  Dowell, Mark & Peter Jarratt (1971). **“A Modified Regula Falsi Method for Computing the Root of an Equation”**. In: *BIT* 11.2, pp. 168–174. DOI: *10.1007/BF01934364*.
-  Snyder, James N. (1953). ***Inverse interpolation, a real root of $f(x) = 0$*** . ILLIAC I Library Routine H1-71. University of Illinois Digital Computer Laboratory, p. 4.

Graphs: <https://www.desmos.com/calculator>